Monitoring
Loki & Grafana

VNTECHIES

Today's Lesson

Centralized Logging

Loki & LogQL

VNTECHIES

Log monitoring concepts

VNTECHIES

The Problem

Without Centralized Logging: Typical Scenario

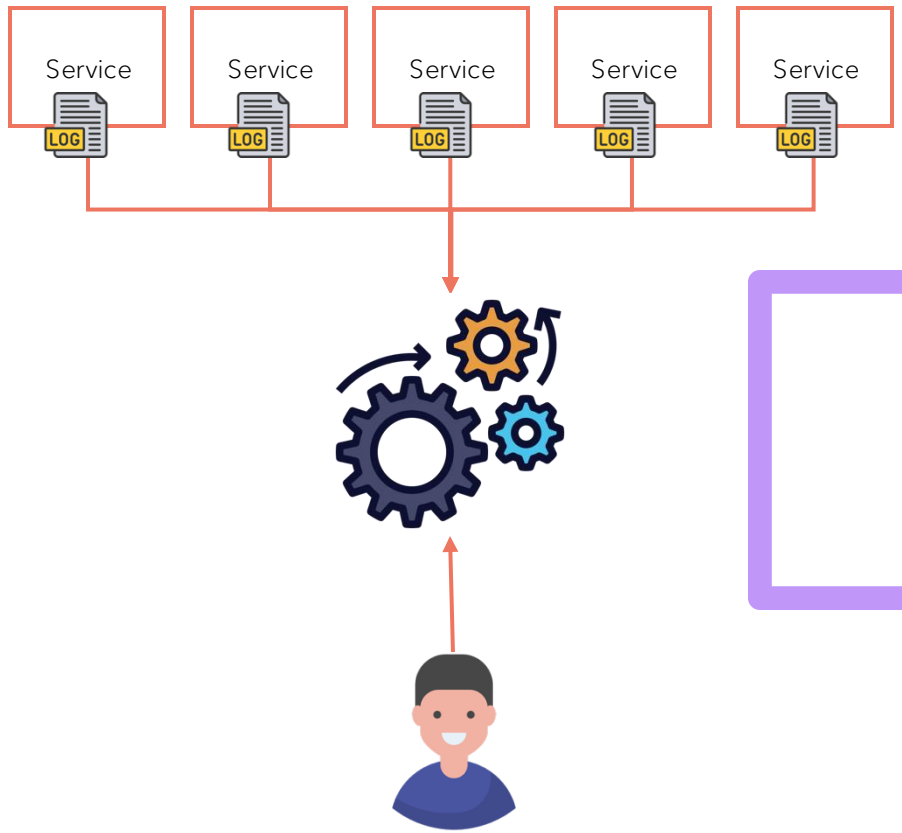
- Each **server**, **container**, **microservice**, or **application** writes logs to its own local file or stdout.
- To investigate issues, engineers must **SSH into multiple machines** or download logs manually.
- There's no automatic way to **aggregate, search, or correlate** logs across syste

The Problem

1. Difficult Troubleshooting
   - When something goes wrong, you must manually check logs on multiple systems.
2. Limited Visibility
   - You only see partial information – if an error happens in another server, you might miss it.
3. No Central Search
   - Want to find all instances of a specific error?
     You'll have to run **separate grep commands** on each host – time-consuming and error-prone.
4. Inconsistent Log Retention
   - Some systems may overwrite or delete logs faster than others.
   - Lost logs = lost context for debugging or auditing later.
5. Poor Security and Compliance
   - You can't easily ensure all logs are captured or securely retained.
   - In regulated industries (finance, healthcare, etc.), this can cause compliance issues.
6. Scaling Issues
   - As you add more servers or containers, the complexity of managing logs grows exponentially.
7. Limited Automation
   - Without centralized analysis, it's hard to set up proactive alerts or detect anomalies automatically.

The Solution

- Centralized logging is the process of collecting log data from various sources across your infrastructure–servers, applications, containers, network devices, and cloud services–and storing it in a single, unified location for analysis and management.
- Instead of having to log into dozens of machines or services individually, engineers can view, search, and analyze all logs in one place.
- Loki is one of the tool help to Centralized logging

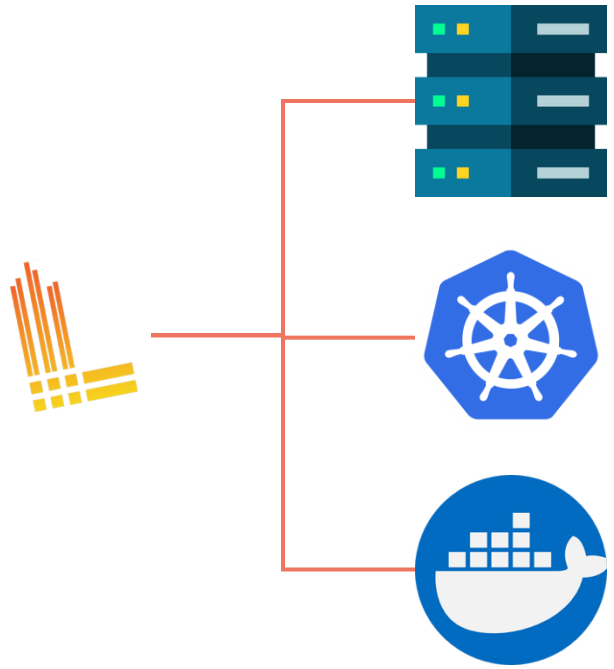


VNTECHIES

Loki

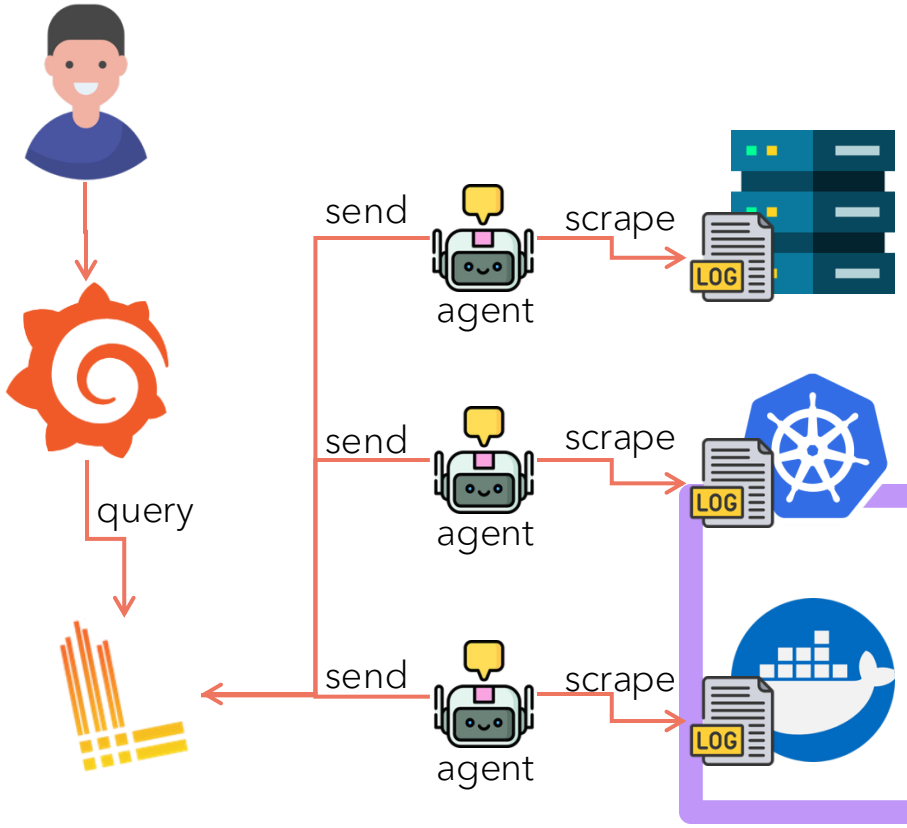
Loki

- Loki is a log aggregation (Centralized logging) system developed by Grafana Labs. It's designed to collect, store, and query logs from multiple sources, similar to what Elasticsearch does in the ELK stack – but with a focus on efficiency, simplicity, and cost-effectiveness.



Loki Architecture

- Promtail (Log Collector)
  - An agent that runs on your servers or containers.
  - It discovers log files (like **/var/log/*** or container logs), attaches metadata (labels), and sends logs to Loki.
- Loki (Log Aggregator and Indexer)
  - The backend service that receives logs from Promtail or other agents.
  - Stores and indexes logs efficiently.
- Grafana (Visualization and Querying)
  - You use Grafana to **search, visualize, and correlate logs** stored in Loki.
  - Queries are written using **LogQL**, a query language similar to PromQL (for Prometheus).



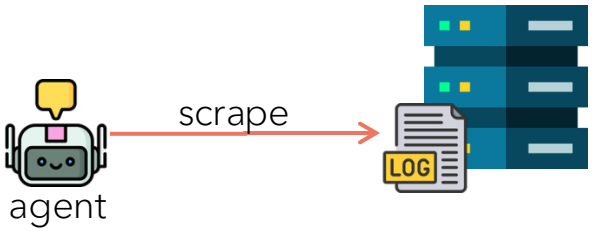
How it work together?

Log Discovery

- Promtail uses **service discovery mechanisms** to find log files automatically.
  Depending on the environment, it supports:
  - **Local file paths** (e.g., **/var/log/syslog**)
  - **Kubernetes pods** (via API or labels)
  - **Systemd journal** (on Linux systems)
- For example, in Kubernetes, Promtail uses the Kubernetes API to find pod labels, namespaces, and container names so you can easily link logs back to the right service.

Log Scraping

- Once Promtail knows where the logs are, it **reads (scrapes)** them line by line – just like Prometheus scrapes metrics endpoints.
  It tracks file offsets to avoid duplication, and supports features like multiline log handling (for stack traces, etc.).



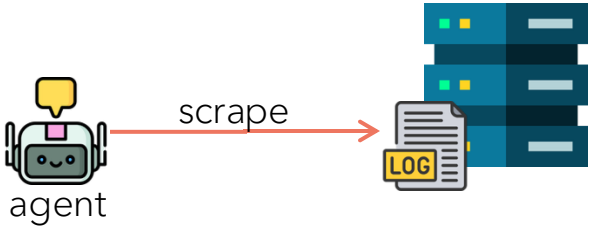
How it work together?

Labeling (Metadata Enrichment)

- Before sending logs, Promtail **attaches labels** – key-value pairs that describe where the log came from.
  For example:
- {job="nginx", host="server1", filename="/var/log/nginx/access.log"}

Pipeline Stages (Processing)

- Promtail includes a configurable **pipeline** to process log lines before sending them.
- Common pipeline stages:
  - **regex**: Extract fields from the log line
  - **json**: Parse structured logs
  - **timestamp**: Parse custom timestamps
  - **labels**: Add or modify labels
  - **output**: Clean and send the final log entry to Loki



job="nginx",
host="server1",
filename="/var/log/nginx/access.log"

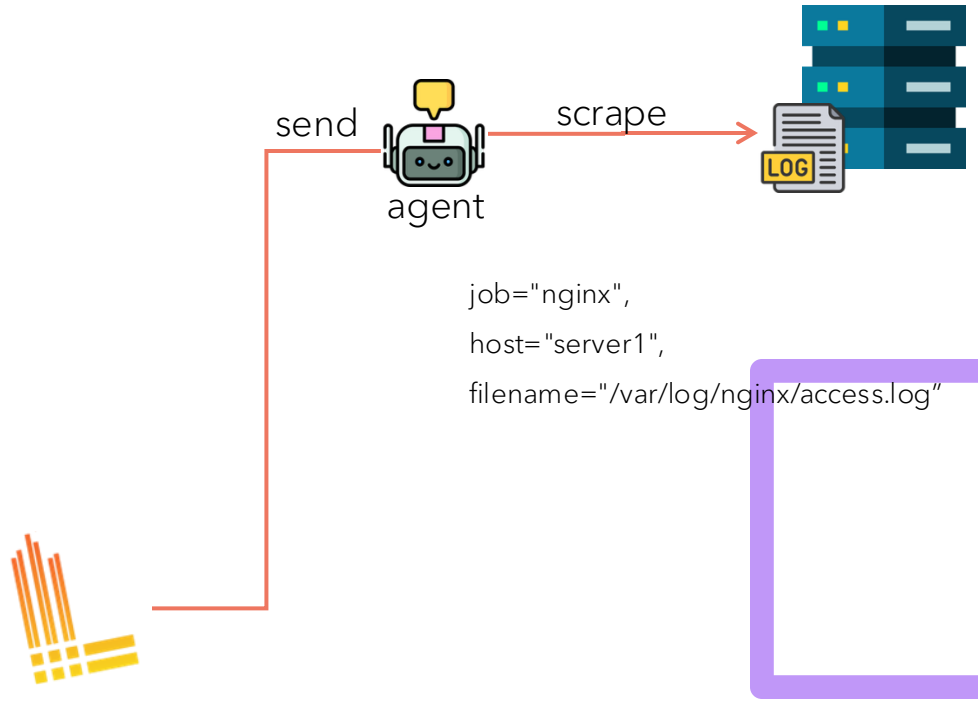
How it work together?

Shipping to Loki

- Once logs are labeled and processed, Promtail sends them to Loki's HTTP endpoint (**/loki/api/v1/push**) in batches.
  Logs are grouped by **stream** (unique combination of labels), then compressed before being sent to minimize network and storage costs.

Monitoring and Reliability

- Promtail retries sending data if Loki is down temporarily.
- It uses **positions files** to remember how far it has read in each file, so it resumes correctly after restarts.



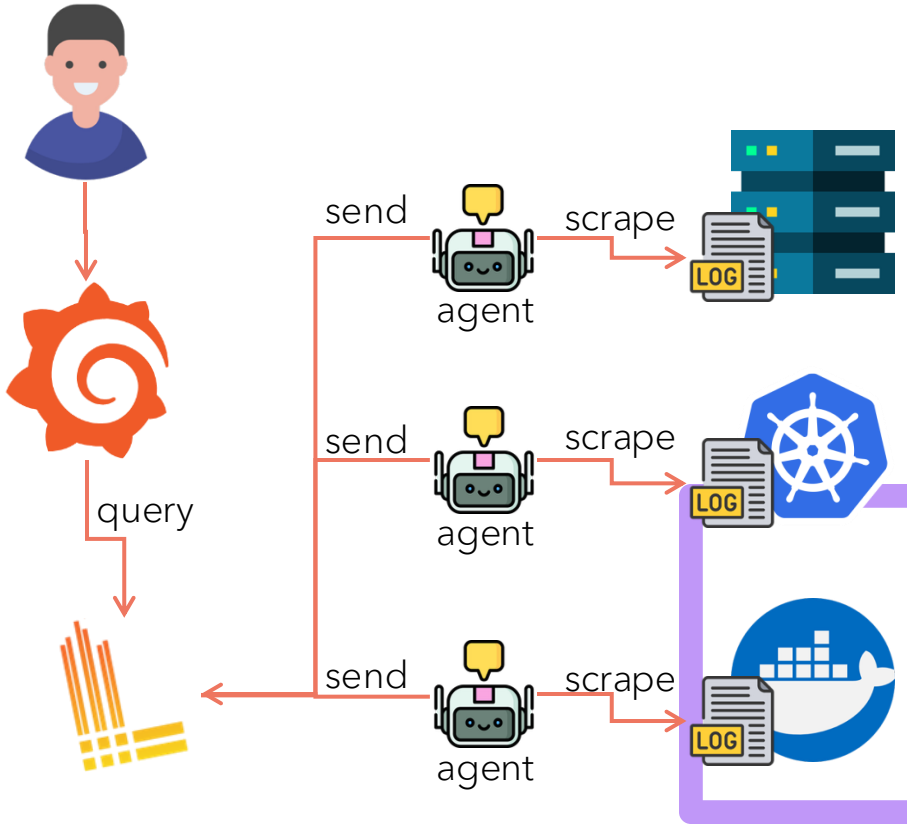
How it work together?

Shipping to Loki

- Once logs are labeled and processed, Promtail sends them to Loki's HTTP endpoint (**/loki/api/v1/push**) in batches.
  Logs are grouped by **stream** (unique combination of labels), then compressed before being sent to minimize network and storage costs.

Monitoring and Reliability

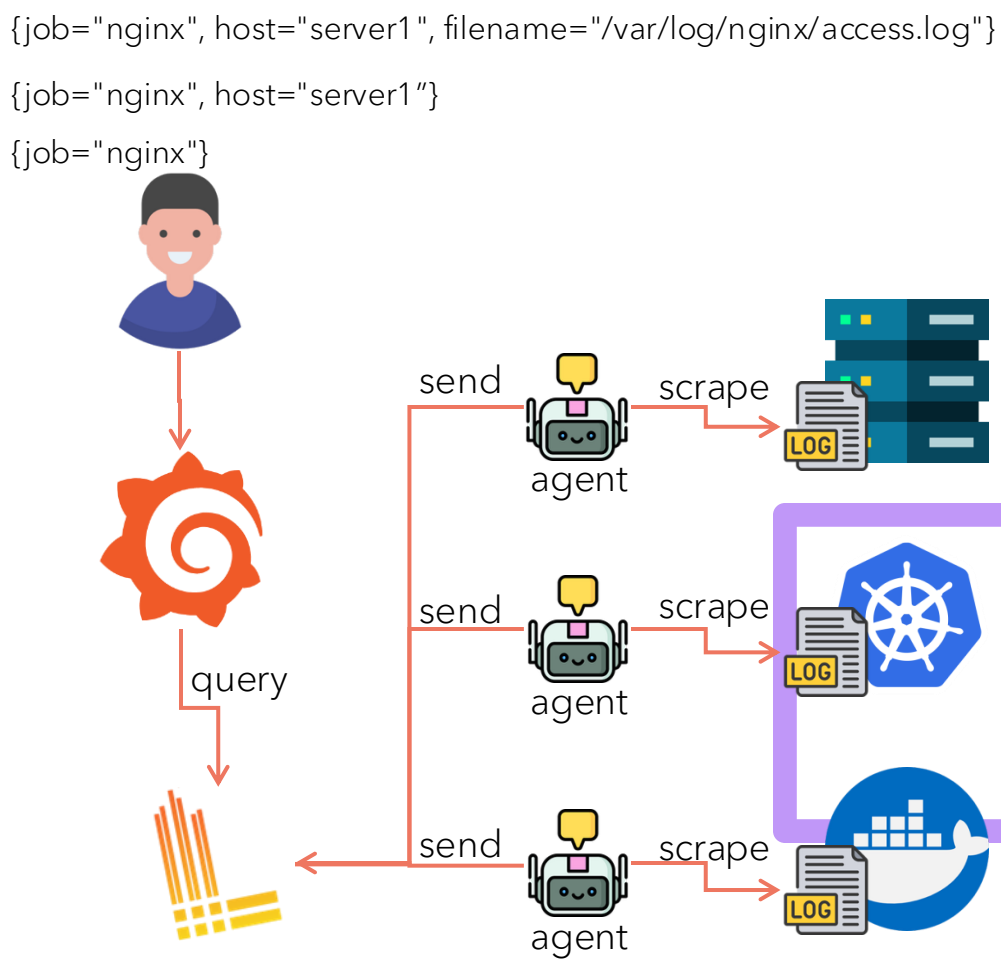
- Promtail retries sending data if Loki is down temporarily.
- It uses **positions files** to remember how far it has read in each file, so it resumes correctly after restarts.



LogQL

- LogQL is Loki's query language used to select and filter log streams, and to build metrics from them. It works by first using a **stream selector** to find log streams based on labels (e.g., {job="my_app"}), and then optionally applying a **filter expression** to narrow down the log lines, such as |= "error" to find lines containing "error"

- LogQL can also convert log streams into metrics using functions like count_over_time.



LogQL

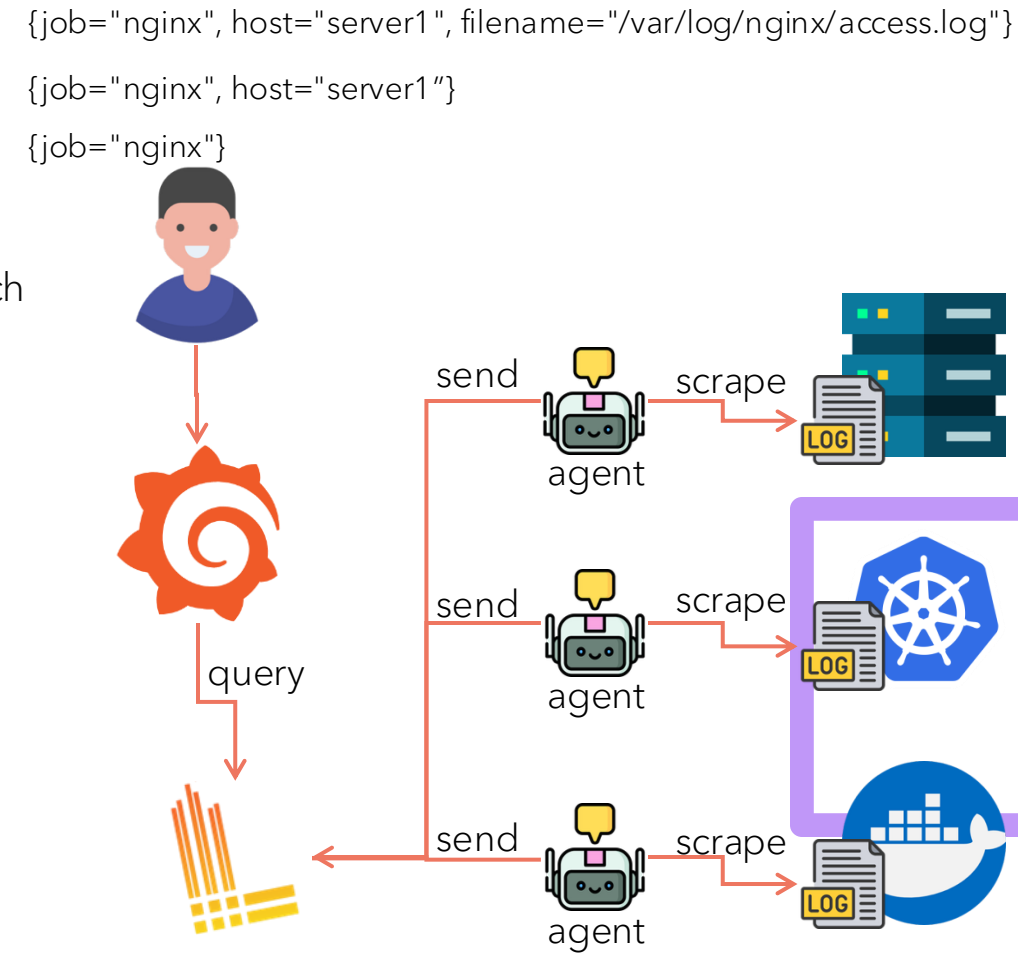
Key components of a LogQL query

Log Stream Selector: This is the required part of a query that specifies which log streams to search for.

- It uses labels enclosed in curly braces, such as {job="resticprofile"}.
- It can include multiple labels to narrow the search, like {job="my_app", namespace="production"}.

Line Filter Expressions: These are optional expressions that filter the log lines within the selected streams.

- They are added after the stream selector using a pipe (|).
- |= "string": Selects lines that contain the specified string.
- != "string": Selects lines that do not contain the specified string.
- |~ "regex": Selects lines that match the specified regular expression.
- !~ "regex": Selects lines that do not match the specified regular expression.
- You can chain multiple filters, and they are executed in order.



VNTECHIES

Practice Lab